COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	**CPE 010 - Data Structures and Algorithms**	**SECTION:**	**CPE21S2**
DATE PERFORMED:	**08-28-26**	**DATE SUBMITTED:**	**09-02-26**
NAME:	**Christian Lee C. Bandigan**	**INSTRUCTOR:**	**Engr. Jimlord M. Quejado**

ACTIVITY NO. 6.1 **Hands-on Activity 6.1 Searching Techniques**

1. PROCEDURE OUTPUT

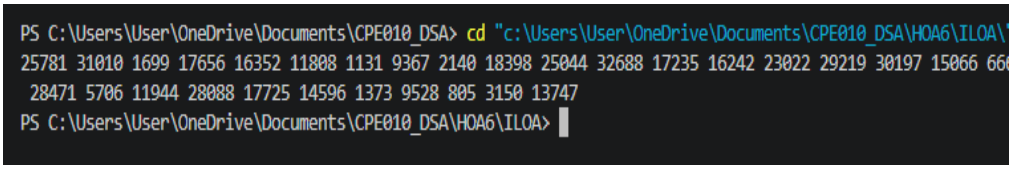
Screenshot	
Observations	This is the generated random integers. This will be the dataset to test the searching algorithms.

Table 6-1. Data Generated and Observations.

Code

```
template <typename T>
void linearSearchAR(int size, T dataFind, T array[]){
//    //while loop
//     int i=0;
//     while(i< size){
//         if(dataFind == array[i]){
//             std::cout <<"Searching is successful\n";
//             return;
//         }
//         i++;
//     }

//     std::cout <<"Searching is unsuccessful\n";

    //for loop implementation
    for(int j=0;j<size; j++){
        if(dataFind == array[j]){
            std::cout <<"Searching is successful\n";
            return;
        }
    }
    std::cout <<"Searching is unsuccessful\n";
}
```

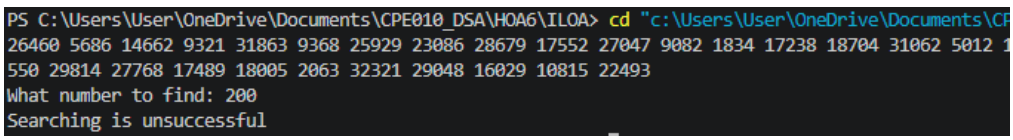
Output	
Observations	A linear search function for an array just uses a loop to compare each element of the array to the element you're trying to find. It can use either a while loop or a for loop but for an array I think the better one to use is a for loop since you already know the mx size.

Table 6-2a. Linear Search for Arrays

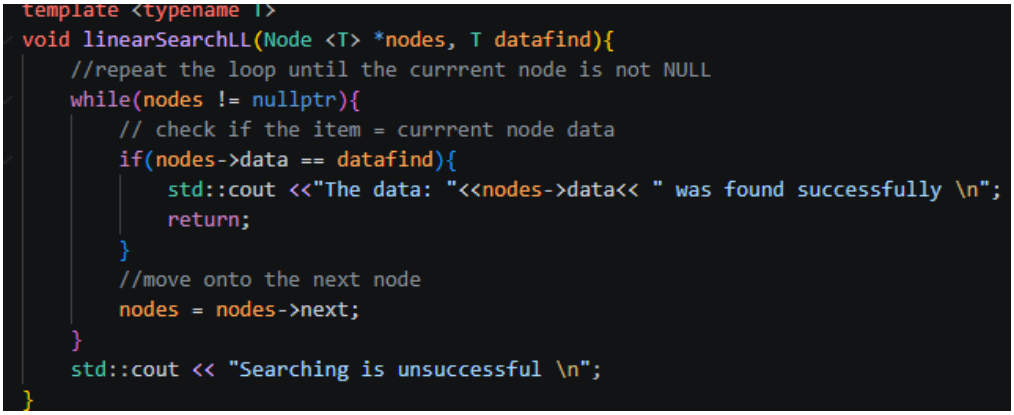
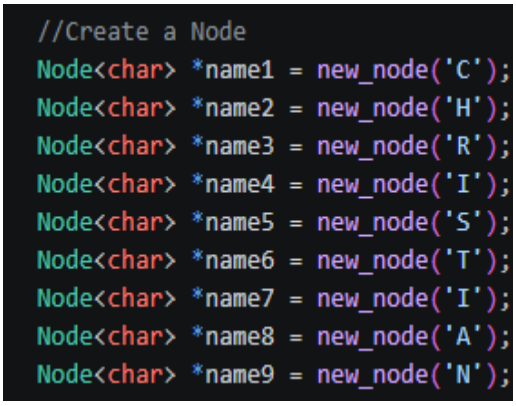
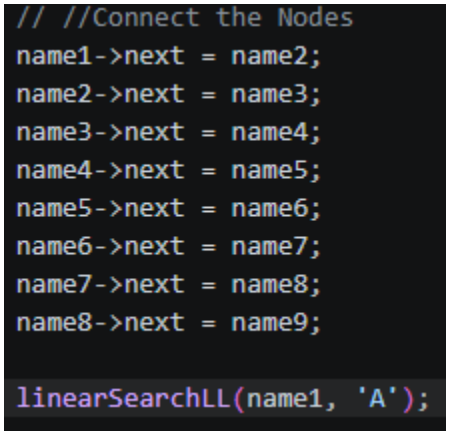
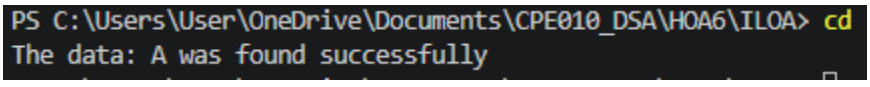
Code	  
Output	
Observations	Basically the same concept for the array but it takes the head node and you need to use a while loop because you don't know how many nodes the linked list has.

Table 6-2b. Linear Search for Linked List

Code

```
45  template <typename T>
46  void binarySearchArr(T data[], int arrSize, T dataFind){
47      int low = 0, high = arrSize-1, middle;
48      while(low <= high){
49          middle = (int)((low+high/2));
50          if(dataFind == data[middle]){
51              std::cout << "the data was found\n";
52              return;
53          }
54          else if(dataFind< data[middle]){
55              high = middle - 1;
56          }
57          else{
58              low = middle + 1;
59          }
60      }
61      std::cout << "the data was not found\n";
62  }
```

```
// //generate a sorted array: (use for loop)
int newArrr[max_size];
for(int i = 0; i <max_size; i++){
    newArrr[i] = i;
}
binarySearchArr(newArrr,max_size,5);
```

Output

```
PS C:\Users\User\OneDrive\Documents\CPE010_DSA\HOA6\ILOA> cd '
the data was found
```

Observations

A binary search for an array uses a while loop to get the middle element, compare it with the element you're trying to find, and based on the comparison it moves the high and low index until the middle element is the element you're trying to find. If you have a sorted array this is faster than linear search because it divides the number of elements to be searched in half each iteration.

Table 6-3a. Binary Search for Arrays

Code

```
64   //middle funnction getter
65   template <typename T>
66   Node<T>* getMiddle(Node<T>* start, Node<T>* last){
67       Node<T>* slow = start;
68       Node<T>* fast = start;
69
70       //continue until fast reaches tthe end or the last node of the currrent search range
71       while(fast != last  && fast-> next != last){
72           //moce slow one step
73           slow = slow->next;
74           //move fast two steps
75           fast = fast->next->next;
76       }
77       return slow;
78   }
```

```
template <typename T>
void binarySearchLL(Node<T>* start, T dataFind){
    //step 1: set start to the  head of the list and set last to null
    Node<T>* last = nullptr;

    while(start != last){
        Node<T>* middle = getMiddle(start, last);

        //if the middle element is same as the key to be searched, we return it.
        if(middle->data == dataFind){
            std::cout << "searching is successful\n";
            return;
        }
        else if(middle->data > dataFind){
            last = middle;
        }
        else{
            start = middle->next;
        }
    }
    std::cout << "Searching is unsuccesssful\n";
}
```

```
Node<int> *name1 = new_node(10);
Node<int> *name2 = new_node(11);
Node<int> *name3 = new_node(12);
Node<int> *name4 = new_node(13);
Node<int> *name5 = new_node(14);
Node<int> *name6 = new_node(15);
Node<int> *name7 = new_node(16);
```

```
//Connect the Nodes
name1->next = name2;
name2->next = name3;
name3->next = name4;
name4->next = name5;
name5->next = name6;
name6->next = name7;

binarySearchLL(name1,10);
```

Output

```
PS C:\Users\User\OneDrive\Documents\CPE010_DSA\HOA6\ILOA> cd
searching is successful
```

Observations	Basically the same for the array but it uses start and last and you need another function to get the middle element since a linked list does not have indexes. The get middle function works by moving slow one at a time and fast two at a time so when fast reaches the end slow is the middle element.

Table 6-3b. Binary Search for Linked List

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Problem 1:

```
const int max_size = 10;

int main(){
    int arr[max_size] = {15, 18, 2, 19, 18, 0, 8, 14, 19, 14};

    linearSearchAR(max_size,18,arr);

    Node<int>* int1 = new_node(15);
    Node<int>* int2 = new_node(18);
    Node<int>* int3 = new_node(2);
    Node<int>* int4 = new_node(19);
    Node<int>* int5 = new_node(18);
    Node<int>* int6 = new_node(0);
    Node<int>* int7 = new_node(8);
    Node<int>* int8 = new_node(14);
    Node<int>* int9 = new_node(19);
    Node<int>* int10 = new_node(14);

    int1->next = int2;
    int2->next = int3;
    int3->next = int4;
    int4->next = int5;
    int5->next = int6;
    int6->next = int7;
    int7->next = int8;
    int8->next = int9;
    int9->next = int10;
```

```cpp
template <typename T>
void linearSearchAR(int size, T dataFind, T array[]){
    int comparison = 0;
    //for loop implementation
    for(int j=0;j<size; j++){
        comparison++;
        if(dataFind == array[j]){
            std::cout <<"Searching is successful\n";
            std::cout << "The data: " << dataFind << " was found after " << comparison << " comparisons. \n";
            return;
        }
    }
    std::cout <<"Searching is unsuccessful\n";
}

template <typename T>
void linearSearchLL(Node <T> *nodes, T datafind){
    int comparison = 0;
    //repeat the loop until the currrent node is not NULL
    while(nodes != nullptr){
        // check if the item = currrent node data
        comparison++;
        if(nodes->data == datafind){
            std::cout <<"The data: "<<nodes->data<< " was found successfully after "<<comparison << " comparisons.\n"
            return;
        }
        //move onto the next node
        nodes = nodes->next;
    }
    std::cout << "Searching is unsuccessful \n";
}
```

Output

```
Searching is successful
The data: 18 was found after 2 comparisons.
The data: 18 was found successfully after 2 comparisons.
```

When using a linear search algorithm in both array and linked list you need to two comparisons to find the key '18'

Problem 2:

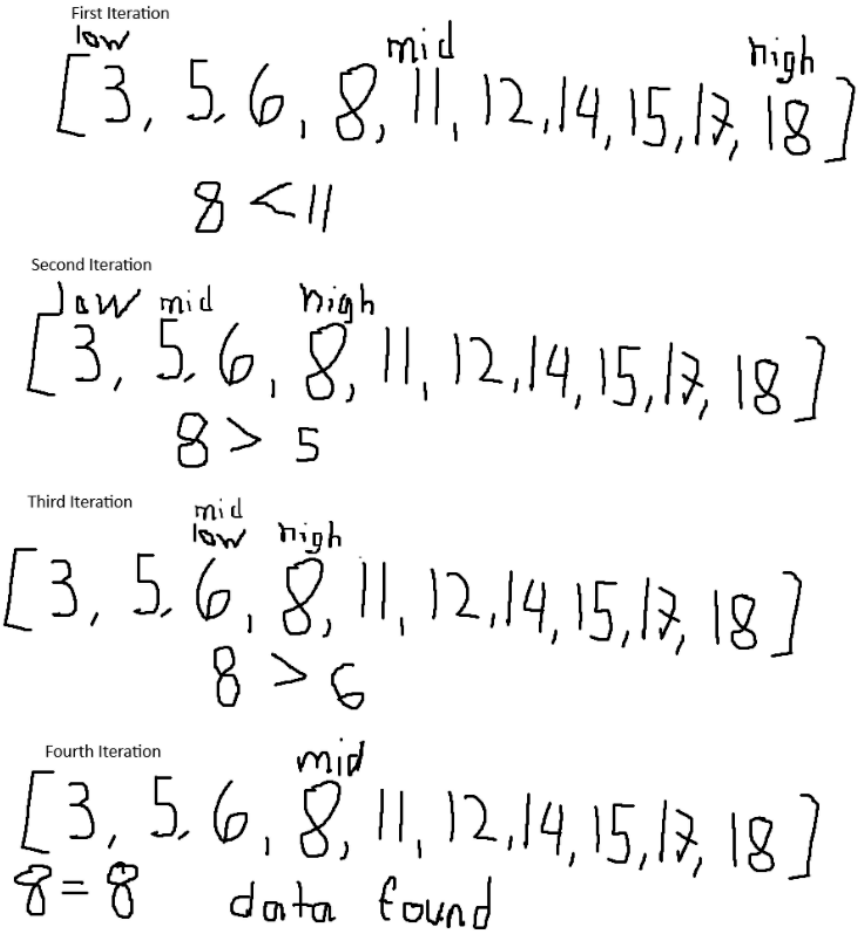
```cpp
template <typename T>
void linearSearchAR(int size, T dataFind, T array[]){
    int repeat = 0;
    bool found = false;
    //for loop implementation
    for(int j=0;j<size; j++){
        if(dataFind == array[j]){
            repeat++;
            found = true;
        }
    }
    if(found == true){
        std::cout << "Searching is successful \n";
        std::cout << dataFind << " was found " << repeat << " time/ s.\n";
    }else{
        std::cout <<"Searching is unsuccessful\n";
    }
}
```

```cpp
template <typename T>
void linearSearchLL(Node <T> *nodes, T datafind){
    int repeat = 0;
    bool found = false;
    //repeat the loop until the currrent node is not NULL
    while(nodes != nullptr){
        // check if the item = currrent node data
        if(nodes->data == datafind){
            repeat++;
            found = true;
        }
        //move onto the next node
        nodes = nodes->next;
    }
    if(found == true){
        std::cout <<"The data: "<<datafind<< " was found successfully. \n";
        std::cout << datafind << " was found " << repeat << " time/s.\n";
    }else{
        std::cout << "Searching is unsuccessful \n";
    }
}
```

Output

```
PS C:\Users\User\OneDrive\Documents\CPE010_DSA\HOA6\Supplementary\ILOB.
Searching is successful
18 was found 2 time/s.
The data: 8 was found successfully.
8 was found 1 time/s.
```

Problem 4:




```cpp
template <typename T>
void binarySearchArr(T data[], int arrSize, T dataFind){
    int low = 0, high = arrSize-1, middle;
    while(low <= high){
        middle = (int((low+high)/2));
        std::cout << "low: " << data[low] << " middle: " << data[middle] << " high: " << data[high] << std::endl;
        if(dataFind == data[middle]){
            std::cout << "the data was found\n";
            return;
        }
        else if(dataFind< data[middle]){
            high = middle - 1;
        }
        else{
            low = middle + 1;
        }
    }
    std::cout << "the data was not found\n";
}
```

Output

```
PS C:\Users\User\OneDrive\Documents\CPE010_DSA\HOA6\Supplementary\ILOB.3>
low: 3 middle: 11 high: 18
low: 3 middle: 5 high: 8
low: 6 middle: 6 high: 8
low: 8 middle: 8 high: 8
the data was found
```

Problem 4:

```
45  template <typename T>
46  bool binarySearchArr(T data[], int high, int low, T dataFind){
47      if(low > high){
48          std::cout << "the data was not found\n";
49          return false;
50      }
51      int middle = (int((low+high)/2));
52      if(dataFind == data[middle]){
53          std::cout << "the data was found\n";
54          return true;
55      }
56      else if(dataFind< data[middle]){
57          return binarySearchArr(data, middle - 1, low, dataFind);
58      }
59      else{
60          return binarySearchArr(data,high, middle + 1, dataFind);
61      }
62  }
```

Output

```
PS C:\Users\User\OneDrive\Documents\CPE010_DSA\HOA6\Supplementary\ILOB.3>
the data was found
```

Conclusion

In this activity I learned how linear search and binary search works. Linear search works by comparing the element you're trying to find to each of the elements of a list while binary search works by dividing the search area in half until the middle element is the element you're trying to find. I was able to implement these in C++ to search elements in both an array and linked lists. Moreover, in the supplementary activity, I was able to solve some problems using these two concepts.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

COMPUTER ENGINEERING

QUEZON CITY

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.